Delft University of Technology
Faculty of Electrical Engineering, Mathematics, and Computer Science

# DSAIT4110 Sequential Decision Making

# Assignment 1: Reinforcement Learning

February 9, 2026

# 1 Introduction

In this practical assignment you will develop some hands-on experience with implementing and applying (basic) reinforcement learning techniques. The assignment consists of two parts. First, we will focus on regular Q-learning. Then, we will apply Q-learning with a deep Q network (DQN) [1].

## 1.1 Coding framework

You should download a `zip` archive from Brightspace. Importantly, it contains the following files:

1. To set up the coding environment:
   (a) `Dockerfile` containing the specification of the coding environment.
   (b) `Makefile` containing targets to manage the coding environment and run the Python code.
   (c) (Optionally) `docker/rl-assignment.tar.gz`, a docker image of the coding environment.
2. For the Q-learning part:
   (a) `simple_grid.py`, the environments for the agent that we will be using.
   (b) `q_learning_main.py`, for the main loop you will need to write to run your experiments.
   (c) `q_learning_skeleton.py`, for the Q-learning agent you will need to implement.
3. For the deep Q-learning part:
   (a) `deep_q_learning_main.py`, contains the main loop you can use to run your experiments.
   (b) `deep_q_learning_skeleton.py`, file for the deep Q-learning agent; a part of the code is already provided but you will need to complete it.

## 1.2 Deliverables

For this assignment you are creating two deliverables, which do not need to be submitted to Brightspace. Feedback is available during the lab session on Feb 19.

1. A `zip` file containing *all* code files with your solutions to the coding exercises. You should upload the same directory as you downloaded from Brightspace, with all parts of the codebase marked with `TODO` completed. There is five coding exercises/five `TODOs` in total.
2. A `pdf` document with your answers to all questions from Sections 2 and 3. Your answers should be structured according to the questions in this document. In other words, we do not expect you to write a proper report (no introduction, conclusions, et cetera). However, please support your answers with data—statistical analyses, tables, plots—that you consider informative.

## 1.3 Setup

For this assignment you will need a Python 3.8 environment with OpenAI Gym installed. To make this setup easy and consistent across operating systems, we provide you with a `Dockerfile` and a `Makefile`. Follow the instructions below to get started.

### 1.3.1 Installing Docker

Skip this section if you already have Docker installed on your system.

**Windows.** First install the Windows Subsystem for Linux Version 2 (WSL2)[1] Docker relies on WSL2 for containerization. Once you have a working WSL2 installation, go ahead and install Docker as well.[2]. You may need to restart your system after the installation is completed.

[1] `https://learn.microsoft.com/en-us/windows/wsl/install`
[2] `https://docs.docker.com/desktop/setup/install/windows-install/`

**MacOS and Linux-based OS.** You can directly follow the installation instructions for MacOS[3] or for your Linux distribution[4]. Please note that depending on your specific distribution, you may need to manually create the Docker group and add your user to it. You may also need to restart your system after the installation is completed.

### 1.3.2 Verifying the installation

You can check if Docker is working correctly by running the following command in a `UNIX` terminal or WSL2 `bash` shell on Windows;

```
docker run hello-world
```

You should see `Hello from Docker!` printed in the console.

Additionally, please ensure that `make` is available in your terminal.[5] It should be available by default on most `UNIX`-based OSes. You can run:

```
make
```

If make is not found, go ahead and install it (`sudo apt install make`).

### 1.3.3 Preparing the coding environment

Once you have everything set up, we strongly recommend that you run:

```
make build
```

from the root of the working directory. This will start building the `rl-assignment` image according to the `Dockerfile` in the `docker/` folder. Only if this does not work for some reason, please download the `rl-assignment` image from Brightspace instead, place it in the `docker/` folder, and run:

```
make import_image
```

Once the image has been successfully built (or imported), you can verify it exists by running:

```
docker images
```

which will list all Docker images available on your system, including (hopefully) `rl-assignment`.

## 1.4 Running the code

We provided you with a `Makefile` that abstracts away long commands and arguments used by Docker. You only need two commands to interact with and run your code:

- `make run_ql` will run `q_learning_main.py`
- `make run_dqn` will run `deep_q_learning_main.py`

Note that upon executing `deep_q_learning_main.py`, the code will create a `recorded_episodes` folder containing the recordings of your agent's episodes. These files will be overwritten every time you run `deep_q_learning_main.py`. You can have a look at the `Makefile` for other targets that have been defined.

[3] https://docs.docker.com/desktop/setup/install/mac-install/
[4] https://docs.docker.com/desktop/setup/install/linux/
[5] https://www.gnu.org/software/make/

# 2 Q-Learning

## 2.1 Description

In the first part of this assignment, we will experiment with the "DrunkenWalk" environments that are located in `simple_grid.py`. This is a simple grid environment entirely based on the code of "FrozenLake".[6]

The story here is as follows. You are finding your way home (`G`) after a great party that was happening at `S`. Unfortunately, due to recreational intoxication (or if you prefer: simply tiredness), you find yourself only moving into the intended direction 80% of the time and perpendicular to that the other 20% of the time. To make matters worse, the local community has cut the budget for pavement (`.`) maintenance, which means that your way home is full of potholes (`H`) that are very likely to make you trip. If you fall into one of the potholes, you are (obviously) magically transported back to the party without getting your hard-earned sleep. We will be working with `theAlley` and `walkInThePark` maps in the assignment, but there are some other maps available for debugging purposes. These two are specified as follows:

```
"theAlley": [
    "S...H...H...G"
]

"walkInThePark": [
    "S.......",
    ".....H..",
    "........",
    "......H.",
    "........",
    "...H...G"
]
```

An episode ends when the agent trips at `H` or when it reaches the goal at `S`. Reaching the goal gives a reward of +10. When moving to a pothole tile, there is a 20% risk of tripping that incurs a broken leg penalty.[7] Before you get started, please take a look at all the files related to this part of the assignment.

The Q-Learning algorithm allows us to estimate the optimal Q function using only trajectories from the MDP obtained by following some policy. Q-Learning with $\epsilon$-greedy exploration acts as follows at time $t$:

1. In the current state $s$ take action $a$ such that $a$ is chosen at random with probability $\epsilon$, and chosen greedily $a = max_{a \in A} Q(s,\ a)$ with probability $1 - \epsilon$.
2. Observe the reward $r$ and the next state $s'$.
3. Update the Q-value as follows:

$$Q^{new}(s,\ a) = (1 - \alpha) \cdot Q^{old}(s,\ a) + \alpha \cdot [r + \gamma max_{a' \in A} Q^{old}(s',\ a')]$$

   Also note that when the episode terminates in $s'$, the update is as follows:

$$Q^{new}(s,\ a) = (1 - \alpha) \cdot Q^{old}(s,\ a) + \alpha \cdot r$$

[6] `https://www.gymlibrary.dev/environments/toy_text/frozen_lake/`, credits to the original authors.
[7] This replaces the probability to move perpendicularly and makes our task a bit easier.

## 2.2 Your tasks

### Preliminaries

(a) **Coding Exercise 1.** Implement a Q-Learning agent with $\epsilon$-greedy action selection by completing the template in `q_learning_skeleton.py`

(b) **Coding Exercise 2.** Implement the loop of your Q-Learning agent by completing the `act_loop(·)` function in `q_learning_main.py`.

(c) **Question 1** Which environment, `walkInThePark` or `theAlley`, do you expect to be more difficult to learn? Why?

#### `WalkInThePark`

We will start with the `walkInThePark` map.

(d) **Question 2** Run the agent for 1000 episodes in the `walkInThePark` map with the following settings: $\epsilon = 0.05$, $\gamma = 0.9$, $\alpha = 0.1$. Does the agent learn an optimal policy? Why (not)? Report the (greedy) policy that your agent learned.

#### `theAlley`

Now we will use `theAlley` map. First, you will compute the optimal Q values, $Q^*$. This can be done using value iteration.

(e) **Question 3** For value iteration, how can you deal with the terminal states?

(f) **Question 4** Calculate (or compute) $Q^*$, the optimal Q-values for `theAlley` map with $\gamma = 0.9$ and `broken_leg_penalty` $= -10$.

(g) **Question 5** Run some experiments for 1000 episodes with $\epsilon = 0.05$, $\gamma = 0.9$, $\alpha = 0.1$, `broken_leg_penalty` $= -10$. Does the agent learn an optimal policy? Why (not)?

(h) **Question 6** Now, calculate (or compute) $Q^*$, the optimal Q-values for `theAlley` map with $\gamma = 0.9$ and `broken_leg_penalty` $= -5$.

(i) **Question 7** Run some experiments for 1000 episodes with $\epsilon = 0.05$, $\gamma = 0.9$, $\alpha = 0.1$, `broken_leg_penalty` $= -5$. Does the agent always learn an optimal policy? Why (not)?

(j) **Coding Exercise 3.** Change the exploration strategy of the agent in a way that allows it to find the optimal solution more often and quicker.

(k) **Question 8** Describe and explain your new exploration strategy. To what extent does it achieve the goal of helping the agent learn the optimal policy?

# 3 Deep Q-Learning

## 3.1 Description

We will use the `LunarLander-v2` environment from Gym.[8] In regular Q-Learning we have a table to look up the Q-value for each state-action pair. In Deep Reinforcement Learning, we instead use function approximation. We define the Q-value as $Q(s,\ a;\ \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ are the parameters of the function approximation (in this case a neural network). In `deep_q_learning_skeleton.py`, a basic version of deep Q-learning has already been implemented. Before you get started, familiarize yourself with this file.

## 3.2 Your tasks

(l) **Question 9** Run `deep_q_learning_main.py` a couple of times. What behavior from the agent do you observe? Does it learn to land safely between the flags?

(m) **Coding Exercise 4.** We will make the agent employ experience replay (see [1]). Complete the class `ReplayMemory` in `deep_q_learning_skeleton.py` and update your QLearner:

  (a) `store_experience(·)` should be called in the function `process_experience(·)`.

  (b) in `process_experience(·)` sample a batch of `self.batch_size` from the replay memory and update the network using this experience.

(n) **Question 10** Again, run `deep_q_learning_main.py` a couple of times. What behavior from the agent do you observe now? Does it learn to land safely between the flags? Did the agent improve compared to Question 8?

(o) **Coding Exercise 5.** Now we will use an additional target network, $Q(s,\ a;\ \boldsymbol{\theta}^{-})$. Initially, our Q-network and the target network will have the same parameters. We will use the target network to provide the estimated future values. That is, we change our target from

$$r + \gamma \cdot max_{a'} Q(s',\ a',\ \boldsymbol{\theta}) - Q(s,\ a,\ \boldsymbol{\theta})$$

to

$$r + \gamma \cdot max_{a'} Q(s',\ a',\ \boldsymbol{\theta}^{-}) - Q(s,\ a,\ \boldsymbol{\theta})$$

and at the end of every episode we set $\boldsymbol{\theta}^{-} = \boldsymbol{\theta}$.

  (a) In `deep_q_learning_main.py` add a target network to the initialization of the QLearner.

  (b) At the end of every episode set $\boldsymbol{\theta}^{-} = \boldsymbol{\theta}$ using `load_state_dict(·)` of `target_network`.

  (c) Change `single_Q_update(·)` and `batch_Q_update(·)` to use Q-value estimation for the next state from the target network.

(p) **Question 11** Again, run `deep_q_learning_main.py` a couple of times. What behavior from the agent do you observe this time? Does the agent learn to land safely between the flags now? Did it improve compared to Question 9?

This is the end of Assignment 1: Reinforcement Learning.

# References


[1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.


[8] `https://www.gymlibrary.dev/environments/box2d/lunar_lander/`